

EmbedSim

Framework for Embedded Control Engineers

A Beginner's Guide

Introduction · Motivation · Conceptual Architecture



https://github.com/vectorsim/embed_sim_project

1 What is EmbedSim?

EmbedSim is an open-source Python framework that lets you design, simulate, and generate production C code for embedded control systems — all from a single, readable Python script. Think of it as a lightweight, programmer-friendly alternative to MATLAB/Simulink, built specifically for the constraints of 32-bit microcontrollers such as the Infineon AURIX TriCore and ARM Cortex-M4.



If you have ever written a block diagram on paper and then manually translated it into C code, EmbedSim automates exactly that pipeline.

With EmbedSim you can:

- Connect signal-processing blocks using a clean `>>` operator — just like Simulink's wire connections.
- Simulate the resulting system in Python with float32 precision, matching typical MCU arithmetic.
- Automatically emit `embedsim_loop.c` and `embedsim_loop.h` — ready-to-compile C source — with one function call.
- Gradually accelerate hot blocks using compiled Cython wrappers, without touching the rest of your Python model.
- Import Modelica/FMU plant models alongside your custom blocks through first-class FMU co-simulation support.

EmbedSim in One Picture

The workflow follows a natural left-to-right pipeline:



2 Motivation — Why EmbedSim Exists

The Problem with Today's Toolchains

Embedded control engineers working on motor drives, power converters, or automotive ECUs face a painful choice:

Approach	What's Good	What's Painful
MATLAB / Simulink	Industry standard, huge library, autocoding	Expensive licence, closed ecosystem, black-box codegen
Hand-written C	Full control, zero overhead	No simulation, error-prone, slow iteration
Pure Python (scipy)	Free, flexible, great for analysis	Cannot generate MCU code, float64 everywhere, no block-diagram structure
EmbedSim	Free & open-source, block diagrams in Python, MCU-grade codegen	Younger ecosystem, community-driven

Design Principles

EmbedSim was designed around five non-negotiable principles that every line of the framework upholds:

MCU Parity First

Default precision is float32 — the native type on most 32-bit controllers. Float64 is opt-in, not the default. This means simulation results match MCU behaviour by design, not by accident.

Dual Backend from Day One

Every block runs in pure Python for fast development and testing, and can be switched to a compiled Cython/C backend for production benchmarking — with a single flag at construction time.

Block Diagram as Code

The >> connection operator turns a Python script into an executable block diagram. Topology is explicit, readable, and version-controllable — no hidden wires in a proprietary binary.

Code Generation, Not an Afterthought

C code emission is a first-class feature. The code generator walks the same block graph used for simulation, ensuring that what you simulate is exactly what runs on hardware.

Open & Affordable

MATLAB Simulink with Embedded Coder costs thousands of dollars per seat per year. EmbedSim is MIT-licensed. An engineer in Colombo or Jaffna can use the full toolchain on a laptop.

3 Conceptual Architecture

EmbedSim is organised into four conceptual layers. Each layer has a single responsibility, and they compose cleanly — you can use layer 1 alone for pure Python simulation, or add all four layers for full MCU code generation.

#	Layer	Responsibility
1	Signal Layer	VectorSignal — the float32 data packet flowing between blocks.
2	Block Layer	VectorBlock — the universal base class. All processing, dynamics, sources, and sinks inherit from here.
3	Simulation Layer	EmbedSim — the topological executor. Discovers block order, drives time, integrates state.
4	CodeGen Layer	LoopGenerator + PYXInspector — walks the graph, inspects .pyx files, emits embedsim_loop.c/h.

Layer 1 — The Signal: VectorSignal

A VectorSignal is simply a named NumPy array. It is the currency that blocks exchange. Wrapping the array in a named object gives every value a human-readable label for debugging and plotting.

```
from embedsim.core_blocks import VectorSignal

# A 3-phase current measurement (float32 by default)
i_abc = VectorSignal([2.1, -1.0, -1.1], name='i_abc')

# Double precision override for a precision-critical calculation
from embedsim.core_blocks import VectorSignal
angle = VectorSignal([0.0], name='theta', dtype=np.float64)
```

Layer 2 — The Block: VectorBlock

VectorBlock is the universal base class. Every element in a simulation — sources, gains, integrators, Clarke transforms, PI controllers, FMU wrappers, sinks — is a VectorBlock subclass.

A block exposes exactly three methods that subclasses override:

Method	Purpose
<code>compute_py(t, dt, inputs)</code>	Pure Python logic — always implement this first.
<code>compute_c(t, dt, inputs)</code>	Cython/C accelerated path — implement once performance matters.
<code>_load_wrapper()</code>	Imports the compiled .pyx extension into self._wrapper.

The `>>` operator wires blocks together in a single line:

```
# Connect a sine source → Clarke transform → Park transform → PI controller → sink
source >> clarke >> park >> pi_ctrl >> sink

# Fan-out: one source feeds two blocks
source >> clarke
source >> observer
```

Layer 3 — The Simulation Engine: EmbedSim

EmbedSim takes a list of sink blocks and discovers the full execution graph by walking backwards through input connections. It then:

- Runs a Depth-First Search to establish a topologically sorted execution order.
- Detects and breaks algebraic loops using VectorDelayEnhanced / LoopBreaker blocks.
- Steps forward in time using either Euler or RK4 integration.
- Calls each block's `compute()` in the correct order at every time step.

```
from embedsim import EmbedSim

sim = EmbedSim(sinks=[sink], dt=1e-4, t_end=0.1, solver='rk4')
sim.run()
```

Layer 4 — Code Generation: LoopGenerator & PYXInspector

When you are ready to deploy to hardware, CodeGenerator walks the same block graph and emits C source files. Two components collaborate:

PYXInspector

A static analyser that reads `.pyx` Cython wrapper files and extracts the C function signatures — number of inputs, number of outputs, required C source files. It populates block metadata (`NUM_INPUTS`, `OUTPUT_SIZE`, `C_SOURCES`, `C_HEADERS`) automatically via the `__init_subclass__` hook in `VectorBlock`. No manual annotation required.

LoopGenerator

Walks the topologically sorted block list and calls `_emit_block()` for each block, producing a single `embedsim_loop.c` file containing a `step()` function that mirrors exactly what the Python simulation does — including register (state) variables, input wiring, and output forwarding.

```
from embedsim.code_generator import CodeGenerator

gen = CodeGenerator(sinks=[sink])
gen.generate_loop()          # writes embedsim_gen/embedsim_loop.c
                             #          embedsim_gen/embedsim_loop.h
```

How the Layers Interact — A Complete Example

The following sketch shows how all four layers combine for a simple gain-and-sum simulation:

```
import numpy as np
from embedsim.core_blocks import VectorSignal
from embedsim.source_blocks import SineSource
from embedsim.processing_blocks import VectorGain, VectorSum
from embedsim.dynamic_blocks import VectorEnd
from embedsim import EmbedSim
from embedsim.code_generator import CodeGenerator

# — Layer 2: define blocks —————
source = SineSource('sine', freq=50.0, amplitude=1.0)
gain = VectorGain('K', gain=2.0)
summer = VectorSum('add', signs=[1, -1])
sink = VectorEnd('output')

# — Layer 2: wire the diagram with >> —————
source >> gain >> summer
source >> summer      # second input to summer
summer >> sink

# — Layer 3: simulate —————
sim = EmbedSim(sinks=[sink], dt=1e-4, t_end=0.02, solver='euler')
sim.run()

# — Layer 4: generate C code —————
gen = CodeGenerator(sinks=[sink])
gen.generate_loop()      # → embedsim_gen/embedsim_loop.c
```

4 Key Abstractions — Quick Reference

Concept	One-liner
VectorSignal	A named float32 NumPy array — the data packet between blocks.
VectorBlock	Abstract base class for every element in the diagram.
compute_py()	Your Python-side block logic. Always implement this first.
compute_c()	Cython-accelerated path. Flip <code>use_c_backend=True</code> to activate.
>> operator	Connects block A's output to block B's input list.
EmbedSim	The time-stepping engine. Owns topology sort, RK4, loop detection.
VectorEnd	Sink block. Records a signal's time history for plotting.
CodeGenerator	Emits <code>embedsim_loop.c</code> / <code>.h</code> from the same graph used for simulation.
PYXInspector	Parses <code>.pyx</code> files to auto-populate C metadata on block classes.
PYX_FILE	Class attribute pointing to a block's <code>.pyx</code> wrapper. Triggers auto-inspection.
C_INPUT_MAP	Dict mapping input port indices to named C register variables.
use_c_backend	Constructor flag: <code>False</code> = Python (default), <code>True</code> = compiled C.
DEFAULT_DTYPE	Module-level dtype (float32). Change once to switch the whole framework.



Tip for beginners: start with `use_c_backend=False` and `DEFAULT_DTYPE=np.float32`. Only add C backends after you have verified the Python model is correct.

5 Next Steps

Now that you understand what EmbedSim is and how it is structured, here is a recommended learning path:

Step 1 — Run the PMSM FOC simulation

Open `pmsm_foc_pwm_smc.py` and run it. Watch the motor speed converge to the reference. Plot the outputs with `plot_helper`.

Step 2 — Trace one block end-to-end

Pick `VectorGain`. Read `compute_py()`. Connect it to a `SineSource` and a `VectorEnd`. Run a 1-second simulation. Verify the output is 2× the input.

Step 3 — Read the topology printer

Call `topology_printer.print_topology(sinks=[...])` on your small model. See how EmbedSim discovers and labels the block graph.

Step 4 — Generate C code

On your small model, call `CodeGenerator(...).generate_loop()`. Open `embedsim_gen/embedsim_loop.c` and read the `step()` function line by line.

Step 5 — Add a new block

Write a `VectorSaturation` wrapper subclass with a `PYX_FILE`. Add `C_INPUT_MAP` and watch `PYXInspector` populate `NUM_INPUTS` automatically.

Step 6 — Build for hardware

Take the generated `.c` and `.h` files, drop them into your AURIX or Cortex-M4 project, wire to your ADC/PWM ISR, and compile with `TASKING` or `GCC`.



EmbedSim is designed so that each layer can be adopted independently. You do not need to use code generation to benefit from block-diagram simulation, and you do not need to use Cython to benefit from code generation.